

Beyond basics:

1) Splitting text to chars $\rightarrow$

$s = \text{"hello"}$

$\text{list}(s) \rightarrow ['h', 'e', 'l', 'l', 'o']$

2) list concatination

$a = [1, 2, 3]$

$a + b = [1, 2, 3, 4, 5, 6]$

$b = [4, 5, 6]$

3) list repetition

$a = [0]$

$a * 5 \rightarrow [0, 0, 0, 0, 0]$

comphrension:

$lis = []$

for i in $\text{range}(1, 21)$:

if $(i \% 2 == 0)$:

$lis.append(i)$



$[i \text{ for } i \text{ in range}(1, 21) \text{ if } i \% 2 == 0]$

Set

① list $\rightarrow$ set

lis = [1, 1, 2, 3, 2, 3, 4, 5, 5, 6]

set(lis) $\rightarrow$ [1, 2, 3, 4, 5, 6]



1) find max element in Array

arr = ⁰10, ¹2, ²999, ³1943, ⁴12, ⁵0, ⁶-2, ⁷-6, ⁸-9

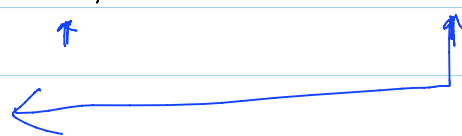
mx = ~~10~~ 999 1943

2) find min element in array

finding 2nd max in array

arr = [5, 15, 10, 6, 14, 7, 9, 15, 14, 14, 15, 15]

0 1 2 3 4 5 6 7 8 9 10 11
[5, 6, 7, 9, 10, 14, 14, 15, 15, 15, 15, 15]



for i in range(len(arr)-2, -1, -1):
 if (arr[i] >= arr[i+1])

[2, 9, 3, 4, 3, 9, 9, 9, 4, 4, 4]

0 1 2 3 4 5 6 7 8 9 10
[2, 3, 3, 4, 4, 4, 4, 9, 9, 9, 9]

[9, 9, 9, 9, 9, 9, 9, 9, 9]

↑

finding Anagram String / no-

s = listen
t = silent } Anagram

```
1 s = "listen" +  
2 t = "silent" -  
3  
4 lis1 = list(s) → 7 -  
5 lis1.sort() → 7 -  
6  
7 lis2 = list(t) → 7 -  
8 lis2.sort() → 7 -  
9  
10 print(lis1) } 7 - 35  
11 print(lis2)
```

lis = []
> for i in s:
 lis.append(s)

$$\text{ord}(e) - \text{ord}(a) \Rightarrow 1$$
$$\text{ord}(n) - \text{ord}(a) \Rightarrow 13$$

Hashing.

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
/	/	/		9				9		10		9						9	9						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

listen
108 105 115 116 101 110

silent
115 105 108 101 110 116

3) Remove Duplicates in Array!

arr = [1, 2, 2, 3, 1, 4, 5, 6, 6]

res

output → [1, 2, 3, 4, 5, 6]

res = [1, 2, 3, 4, 5, 6]

arr = [1, 2, 2, 3, 1, 4, 5, 6, 6]



res = []

for i in arr:

if i not in res

res.append(i)

Lambda functions

```
def add(x, y):  
    return x+y
```

```
res = add(10, 20)  
print(res)
```

```
add = lambda x, y : x+y
```

```
print(add(10, 20))
```

```
var = lambda p1, p2, p3 : expression
```

→ by default the lambda function returns the expression directly.

⑤ find elements which repeated only once

inp $\rightarrow$ [5, 8, 7, 6, 7, 7, 9, 5, 5, 8, 10, 11]

op $\rightarrow$ [6, 9, 10, 11]

```
{  
  5: 3  
  8: 1  
  7: 3  
  6: 1  
  9: 1  
  10: 1  
  11: 1  
}
```

Steps to solve

- ① count freq using dictionary
- ② Iterate keys and get only keys having count == 1

5 1 5

```
{  
  5: 1  
}
```

freq.get(10)

```

if(freq.get(i) == None):
    freq[i] = 1
else:
    freq[i] += 1

```

← Same → `freq[i] = freq.get(i,0)+1`

{
2:10
4:2
}

→ if there is no key in dictionary it returns default value
→ else the existing value

```

1 def findSingles(arr):
2     freq = dict()
3     for i in arr:
4         if(freq.get(i) == None):
5             freq[i] = 1
6         else:
7             freq[i] += 1
8     res = []
9     for i in freq.keys():
10        if(freq[i] == 1):
11            res.append(i)
12    print(res)
13 res = findSingles([1,1,2,5,4,7,8,2,1,4,5,6,3,7])

```

} → counting frequencies of elements

} → taking elements whose having a count of 1

Learning map of DSA for placement prep Next 6 months

suggestion

→ pick (C, C++, java, python)

→ 1 month dedicate for language proficiency

→ daily 2-3 problems x 60 days → (120 - 180) problems

DSA (theory)

Linear DSA

- Arrays
- Linked-List
- Stack
- Queue
- String (optional)

Non-Linear DSA

- tree
 - Binary tree
 - Binary Search tree
 - AVL
 - Heap (min/max)
- Graph
 - Connected
 - Not connected
 - weighted / unweighted
 - directed / un-

(Problem Solving Map)

- 1) Loops
- 2) Math using code (square, root, pow, prime)
- 3) Bit manipulation

- 4) Arrays
 - two pointers
 - sliding window
 - prefix sum
 - kadane's algorithm

- 5) Strings
 - Hashmap / dictionary
 - two pointer
 - sliding window

6) Linked List

- slow / fast pointer
- two pointers

7) Stack

8) Recursion

- linear
- Non-linear
- Subsets
- Include exclude
- Backtracking

eat well code well

😊

Good luck